

Python pour la data

Rappels Python, puis un tableau de 800 Pokémon dans pandas : filtrer, trier, compter, regrouper, et trois graphiques qui répondent chacun à une question.

ON REPART AVEC · Un notebook avec 3 questions posées sur un dataset et 3 graphiques qui y répondent, poussé sur GitHub à la séance 3.

01 UN CSV, C'EST DU TEXTE 02 CINQ VERBES 03 REGROUPER
04 UN GRAPHIQUE RÉPOND À UNE QUESTION 05 ENTRE DEUX SÉANCES

À LA FIN DE LA SÉANCE

Tu sauras lire un DataFrame avec head, shape et describe, et dire ce qu'est une ligne, une colonne, un index.

Tu sauras filtrer, trier, compter et regrouper un tableau en une ligne de pandas chacune.

Tu sauras choisir entre barres, nuage de points et histogramme selon la question posée.

Un CSV, c'est du texte

Le notebook charge 800 Pokémon depuis un fichier CSV. Ouvre-le dans un éditeur de texte avant pandas : ce n'est que du texte, avec des virgules.

pokemon.csv, ouvert dans un éditeur de texte

du texte, des virgules, une ligne par Pokémon

```
#,Name,Type 1,Type 2,Total,HP,Attack,...
1,Bulbasaur,Grass,Poison,318,45,49,...
2,Ivysaur,Grass,Poison,405,60,62,...
3,Venusaur,Grass,Poison,525,80,82,...
```

→ `pd.read_csv(url)` lit ce texte et fabrique le tableau

df, le DataFrame

les mêmes valeurs, rangées et numérotées

index		en-tête						
idx#		Name	Type 1	Type 2	Total	HP	Attack	
0	1	Bulbasaur	Grass	Poison	318	45	49	
1	2	Ivysaur	Grass	Poison	405	60	62	← ligne
2	3	Venusaur	Grass	Poison	525	80	82	

colonne ↑

800 lignes × 13 colonnes : `df.shape` donne (800, 13)
une colonne seule, `df["Speed"]`, est une Series

Le CSV est du texte avec des virgules ; pandas en fait un tableau qu'on interroge.

Devant n'importe quel tableau, trois réflexes, dans cet ordre :

```
df.head()          # les 5 premières lignes
df.shape           # (800, 13) : 800 lignes, 13 colonnes
df.describe()      # min, max, moyenne de chaque colonne de nombres : vitesse max 180
```

- LES RAPPELS PYTHON EN UNE CARTE

Tout ce dont on a besoin en data tient en quatre briques.

```
vitesses = [45, 65, 43, 90, 110]          # liste : plusieurs valeurs
dans l'ordre
pikachu = {"nom": "Pikachu", "type": "Electric", "vitesse": 90}  # dictionnaire :
clé → valeur, une fiche
for v in vitesses: print(v)                # boucle : on passe sur
chaque valeur
def moyenne(liste): return sum(liste) / len(liste)  # fonction : une recette
réutilisable
```

Une liste de dictionnaires, c'est déjà presque un tableau : pandas fait le reste.

Cinq verbes

Un verbe, une cellule exécutée, une question posée au groupe. Le tableau sert de fiche à retrouver ensuite.

VERBE	LA QUESTION QU'IL POSE	LA LIGNE PANDAS	EXEMPLE POKÉMON
Filtrer	Lesquels respectent une condition ?	<code>df[df["Type 1"] == "Fire"]</code>	52 Pokémon de type Feu
Combiner	Deux conditions à la fois ?	<code>df[(df["Speed"] > 100) & (df["Attack"] > 100)]</code>	38 rapides et forts
Trier	Qui est premier ?	<code>df.sort_values("Speed", ascending=False)</code>	DeoxysSpeed Forme, 180 de vitesse
Compter	Combien de chaque ?	<code>df["Type 1"].value_counts()</code>	Water, 112 Pokémon
Regrouper	Une moyenne par catégorie ?	<code>df.groupby("Type 1")["Total"].mean()</code>	Dragon, 550,5 en moyenne

Regrouper

La question du jour : quel type est le plus puissant ?

1. les lignes, mélangées

df

Charmander	Fire	309
Charmeleon	Fire	405
Caterpie	Bug	195
Metapod	Bug	205
Dratini	Dragon	300
Dragonair	Dragon	420

2. un tas par type

.groupby("Type 1")

Dragon

Dratini	300
Dragonair	420

Fire

Charmander	309
Charmeleon	405

Bug

Caterpie	195
Metapod	205

3. une moyenne par tas

["Total"].mean()

Dragon 550,5
1er sur 18 types · 32 Pokémon

Fire 458,1
5e sur 18 types · 52 Pokémon

Bug 378,9
18e sur 18 types · 69 Pokémon

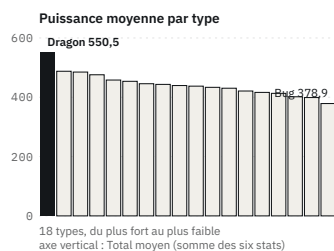
Une ligne de code, une réponse : le type le plus puissant est Dragon.

Regrouper, c'est faire des tas puis calculer une chose par tas.

Un graphique répond à une question

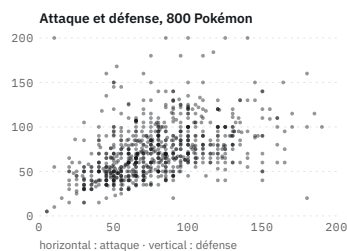
Pas de graphique sans titre ni axes : sans eux, personne ne sait à quelle question il répond.

BARRES · COMPARER DES CATÉGORIES



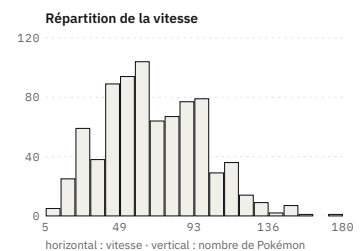
Quel type est le plus puissant ?
Dragon, loin devant ; Insecte ferme la marche.

NUAGE DE POINTS · UN LIEN ENTRE DEUX NOMBRES ?



Les Pokémon forts en attaque sont-ils solides en défense ? Un peu : le nuage monte, sans s'aligner (corrélation 0,44).

HISTOGRAMME · COMMENT UNE VALEUR SE RÉPARTIT



Comment la vitesse se répartit-elle ? Une bosse entre 40 et 110, et quelques fusées au-delà de 150.

QUEL GRAPHIQUE ? · SUR PAPIER

Pour chaque question, choisis le graphique qui y répond.

Barres : comparer des catégories

Nuage de points : chercher un lien entre deux nombres

Histogramme : voir comment une valeur se répartit

1. Quel type de Pokémon est le plus fréquent ?
2. Les Pokémon forts en attaque sont-ils aussi solides en défense ?
3. Comment se répartissent les points de vie ?
4. Quelle génération a les Pokémon les plus rapides, en moyenne ?
5. Un Pokémon rapide est-il fragile ?
6. Les vitesses sont-elles serrées autour d'une valeur, ou très étalées ?

Les réponses sont dans le corrigé, en fin de leçon.

À toi : 3 questions, 3 graphiques

Ton premier projet de portfolio, en 25 minutes. Choisis trois questions qui t'intéressent vraiment sur les Pokémon ; pour chacune, une cellule de calcul, un graphique qui y répond, et la réponse en une phrase sous le graphique. Si tu bloques sur la syntaxe, demande le squelette de la ligne, pas la réponse.

Entre deux séances

Le carnet `02_exercices.ipynb` : 12 exercices, de ★ à ★★★★★, avec vérification automatique, indices et solutions repliées. Il s'ouvre par un quiz de trois questions, celles qu'on se pose à voix haute au début de la séance suivante.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

1. Le notebook affiche (800, 13) quand tu tapes `df.shape`. Ça veut dire quoi ?
 - A. 800 colonnes et 13 lignes
 - B. 800 lignes et 13 colonnes
 - C. 800 Pokémon dont 13 légendaires
 - D. Une erreur : il manque une colonne
2. Tu veux savoir quel type de Pokémon a la meilleure vitesse moyenne. Quelle ligne ?
 - A. `df.sort_values("Speed")`
 - B. `df["Type 1"].value_counts()`
 - C. `df.groupby("Type 1")["Speed"].mean()`
 - D. `df[df["Speed"] > 100]`

3. Quelle ligne garde les Pokémon à la fois rapides (Speed > 100) et forts (Attack > 100) ?
- A. `df[df["Speed"] > 100 and df["Attack"] > 100]`
 - B. `df[(df["Speed"] > 100) & (df["Attack"] > 100)]`
 - C. `df[df["Speed"] > 100 & df["Attack"] > 100]`
 - D. `df.sort_values(["Speed", "Attack"])`
4. Un graphique en barres, sans titre ni nom d'axe, montre trois barres de hauteurs différentes. Que peut-on en conclure ?
- A. Que la première catégorie est la meilleure
 - B. Que les données sont fausses
 - C. Rien : on ne sait ni ce qui est compté, ni dans quelle unité
 - D. Qu'il faut plus de barres

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Un CSV est du texte avec des virgules ; pandas en fait un DataFrame, un tableau qu'on interroge.
2. Trois réflexes devant un tableau inconnu, `head()`, `shape`, `describe()`.
3. Cinq verbes, filtrer, combiner, trier, compter, regrouper, une ligne chacun.
4. `groupby` fait des tas par catégorie, puis calcule une chose par tas.
5. Un graphique répond à une question, et il a toujours un titre et des axes.

Corrigé

Quel graphique ?

1. **Barres** · On compare des catégories : une barre par type, la plus haute gagne. Ici Water, 112 Pokémon.
2. **Nuage de points** · Deux nombres par Pokémon, attaque et défense : un point chacun, et on regarde si le nuage monte. Il monte un peu, corrélation 0,44.
3. **Histogramme** · Une seule valeur, les HP, et on veut voir la forme : où sont la plupart des Pokémon, où sont les rares. La moitié a 65 HP ou moins, un seul monte à 255.
4. **Barres** · Une moyenne par génération, six catégories à comparer : des barres. La première génération est en tête, 72,6 de vitesse moyenne.
5. **Nuage de points** · Vitesse contre défense, deux nombres : un nuage de points. S'il descendait nettement, les rapides seraient fragiles ; il est très dispersé.
6. **Histogramme** · Une seule valeur, la vitesse, et on veut voir sa forme. La moitié centrale des Pokémon tient entre 45 et 90, mais la queue court jusqu'à 180 : étalée vers la droite.

Mini-quiz

1. **B. 800 lignes et 13 colonnes** · shape donne toujours (lignes, colonnes) dans cet ordre. 800 Pokémon, 13 mesures par Pokémon. Les légendaires, eux, sont 65 : ça se compte avec un filtre, pas avec shape.
2. **C. `df.groupby("Type 1")["Speed"].mean()`** · « Par type » appelle groupby : un tas par type, puis la moyenne de la vitesse dans chaque tas. Trier donne les Pokémon un par un, value_counts compte les types, le filtre garde les rapides sans les regrouper.
3. **B. `df[(df["Speed"] > 100) & (df["Attack"] > 100)]`** · pandas relie deux conditions avec &, chacune entre ses parenthèses. Sans elles, Python calcule d'abord `100 & df["Attack"]` et la ligne plante ; « and » ne sait pas comparer deux colonnes entières et plante aussi ; `sort_values` range, il ne filtre pas. La bonne ligne garde 38 Pokémon.
4. **C. Rien : on ne sait ni ce qui est compté, ni dans quelle unité** · Sans titre ni axes, on ne sait pas à quelle question le graphique répond, ni ce que mesure la hauteur. C'est la règle de la séance : pas de graphique sans titre ni axes.